NEXO

Sistema Planeador de Horarios

Documento de Arquitectura Backend

Monolito Modular con Spring Boot 3

SaaS Kumorai
Versión 1.0 | Marzo 2026

Tabla de Contenido

Tabla de Contenido 2
1. Framework: Spring Boot 3 3
1.1 Stack tecnológico completo 3
2. Arquitectura: Monolito Modular 3
2.1 Justificación 4
2.2 Principios de diseño 4
3. Módulos del Sistema 4
4. Generación del Proyecto Inicial 5
4.1 Spring Initializr 5
4.2 Dependencias iniciales 5
4.3 Comando de generación 6
5. Distribución de Carpetas 6
5.1 Estructura raíz del proyecto 7
5.2 Paquete principal: com.kumorai.nexo 7
5.3 Directorio resources 11
5.4 Directorio de tests 11
6. Convenciones y Reglas del Proyecto 11
6.1 Reglas de comunicación entre módulos 11
6.2 Convenciones de nombrado 12
6.3 Estructura de URLs de la API 12
7. Perfiles de Configuración 12
8. Mapa de Dependencias entre Módulos 13
9. Primeros Pasos para el Equipo 13

(Actualizar tabla de contenido al abrir en Word)

1. Framework: Spring Boot 3

El framework seleccionado para el backend de Nexo es Spring Boot 3 (con Spring Framework 6), ejecutado sobre Java 17 LTS. Esta decisión se fundamenta en los siguientes criterios:

Madurez y ecosistema: Spring Boot es el framework más utilizado en el ecosistema Java empresarial. Cuenta con documentación extensa, una comunidad activa masiva y compatibilidad nativa con prácticamente cualquier librería Java que el proyecto necesite.

Arquitectura modular nativa: Spring soporta de forma natural la organización del código en módulos desacoplados dentro de un mismo artefacto desplegable, lo cual es exactamente lo que requiere el enfoque de monolito modular definido para Nexo.

Inyección de dependencias: El contenedor IoC de Spring gestiona automáticamente el ciclo de vida de los componentes, eliminando la necesidad de instanciar manualmente servicios, repositorios y controladores.

Seguridad integrada: Spring Security proporciona autenticación basada en JWT, control de acceso por roles (RBAC), protección CSRF, y cifrado de contraseñas con BCrypt, cubriendo directamente los requerimientos RF-07, RF-08 y RF-56.

Persistencia flexible: Spring Data JPA permite trabajar con bases de datos relacionales mediante repositorios declarativos, mientras que la elección entre SQL y NoSQL (RNF-12) puede diferirse sin impacto en la arquitectura de servicios.

Herramientas de productividad: Spring Boot DevTools, Actuator para monitoreo, y soporte nativo para perfiles de configuración (dev, staging, prod) aceleran el desarrollo y operación.

1.1 Stack tecnológico completo

Capa	Tecnología	Versión
Lenguaje	Java (LTS)	17
Framework	Spring Boot	3.3.x
Seguridad	Spring Security + JWT (jjwt)	6.x / 0.12.x
Persistencia	Spring Data JPA + Hibernate	6.4.x
Base de datos	PostgreSQL	16+
Migraciones	Flyway	10.x
Validación	Jakarta Bean Validation	3.0
Documentación API	SpringDoc OpenAPI (Swagger)	2.x
Build	Maven	3.9+
Testing	JUnit 5 + Mockito	5.10 / 5.x
Envío de correo	Spring Boot Mail (JavaMail)	Integrada

2. Arquitectura: Monolito Modular

Nexo adopta una arquitectura de monolito modular. Esto significa que todo el código backend se despliega como un único artefacto JAR, pero internamente está organizado en módulos funcionales con fronteras claras y dependencias controladas.

2.1 Justificación

Equipo de 4 personas: un equipo pequeño no necesita la complejidad operativa de microservicios (orquestación de contenedores, service discovery, trazabilidad distribuida). Un monolito modular permite trabajar en módulos independientes sin esa sobrecarga.

Despliegue único: un solo JAR simplifica enormemente el pipeline de CI/CD, el hosting (un solo servidor o contenedor) y la gestión de costos, algo crítico dado que el hosting lo asume el equipo fundador (RNF-13).

Base de datos compartida: todos los módulos acceden a la misma instancia de base de datos, eliminando la necesidad de sincronización entre servicios y simplificando las transacciones que cruzan dominios.

Escalabilidad futura: si en el futuro algún módulo requiere escalarse independientemente, la separación interna por paquetes facilita la extracción a un microservicio sin reescribir lógica de negocio.

2.2 Principios de diseño

Cada módulo sigue una estructura de tres capas (Controller → Service → Repository) y se comunica con otros módulos exclusivamente a través de interfaces de servicio, nunca accediendo directamente a repositorios ajenos. Las entidades JPA de cada módulo se agrupan en su propio subpaquete, y las relaciones entre entidades de distintos módulos se manejan mediante claves foráneas (IDs), no asociaciones JPA directas, para mantener el bajo acoplamiento.

3. Módulos del Sistema

El sistema se organiza en los siguientes módulos funcionales, cada uno con responsabilidades claramente delimitadas:

Módulo	Responsabilidad	Requerimientos clave
auth	Registro, login, verificación por código, gestión de tokens JWT, cifrado de contraseñas.	RF-01 a RF-08, RF-10
user	Gestión de perfil (apodo, eliminación de cuenta), roles y permisos. Elevación a Usuario Especial.	RF-03 a RF-06
academic	Motor de planeación: oferta académica, asignaturas, grupos, bloques de horario, validación de restricciones (créditos, prerrequisitos, cruces), planes de estudio.	RF-25 a RF-40, RF-44
schedule	Horarios del estudiante: creación, guardado (máx. 5), histórico, notas generales, exportación.	RF-37, RF-38, RF-42, RF-43, RF-54

campus	Sedes, laboratorios, fotos de salones, cálculo de tiempo de desplazamiento, advertencias de traslado.	RF-20 a RF-24, RF-45 a RF-49
content	Avisos (facultad y universitarios, asambleas), bienestar (apoyo alimentario, becas, salud mental), calendario general (festivos, paros).	RF-11 a RF-19, RF-14
report	Recepción y gestión de reportes de errores e inconsistencias por parte de los usuarios.	RF-09, RF-49
admin	Carga de oferta académica, validación de archivos, gestión de contenido universitario, logs de seguridad.	RF-57, RF-58
shared	Componentes transversales: DTOs comunes, excepciones globales, utilidades, configuración general.	Transversal

4. Generación del Proyecto Inicial

4.1 Spring Initializr

El proyecto base se genera desde Spring Initializr (https://start.spring.io) con la siguiente configuración:

Parámetro	Valor
Project	Maven
Language	Java
Spring Boot	3.3.x (latest stable)
Group	com.kumorai.nexo
Artifact	nexo-backend
Name	nexo-backend
Package name	com.kumorai.nexo
Packaging	Jar
Java	17

4.2 Dependencias iniciales

Al generar el proyecto desde Initializr, seleccionar las siguientes dependencias:

Dependencia	Propósito
Spring Web	API REST (controladores, Jackson, Tomcat embebido)
Spring Data JPA	Capa de persistencia con Hibernate
Spring Security	Autenticación, autorización, protección de endpoints

Spring Boot Mail	Envío de códigos de verificación al correo institucional
Spring Validation	Validación de DTOs con Jakarta Bean Validation
PostgreSQL Driver	Driver JDBC para PostgreSQL
Flyway Migration	Migraciones versionadas de base de datos
Spring Boot DevTools	Recarga automática en desarrollo
Spring Boot Actuator	Endpoints de monitoreo y salud del servicio
Lombok	Reducción de boilerplate (getters, setters, builders)

Dependencias adicionales que se agregan manualmente al pom.xml después de generar:

Dependencia (manual)	Propósito
io.jsonwebtoken:jjwt-api/impl/jackson	Generación y validación de tokens JWT
org.springdoc:springdoc-openapi-starter-webmvc-ui	Documentación Swagger UI automática
org.mapstruct:mapstruct	Mapeo automático entre entidades y DTOs

4.3 Comando de generación

Una vez configurado Spring Initializr, se descarga el archivo .zip, se descomprime y se abre en el IDE. Alternativamente, se puede generar vía línea de comandos con curl:

```
curl https://start.spring.io/starter.zip \
  -d type=maven-project \
  -d language=java \
  -d bootVersion=3.3.6 \
  -d groupId=com.kumorai.nexo \
  -d artifactId=nexo-backend \
  -d name=nexo-backend \
  -d packageName=com.kumorai.nexo \
  -d javaVersion=17 \
  -d dependencies=web,data-jpa,security,mail,validation,\
    postgresql,flyway,devtools,actuator,lombok \
  -o nexo-backend.zip

unzip nexo-backend.zip -d nexo-backend
cd nexo-backend
```

5. Distribución de Carpetas

La siguiente estructura representa la organización completa del proyecto backend. Cada módulo funcional contiene sus propias capas (controller, service, repository, entity, dto) para mantener la cohesión y el bajo acoplamiento entre dominios.

5.1 Estructura raíz del proyecto

```
nexo-backend/
├── pom.xml
├── mvnw / mvnw.cmd
├── .mvn/
├── .gitignore
├── README.md
├── docker-compose.yml
└── src/
    ├── main/
    │   ├── java/com/kumorai/nexo/
    │   └── resources/
    └── test/
        └── java/com/kumorai/nexo/
```

5.2 Paquete principal: com.kumorai.nexo

```
com.kumorai.nexo/
├── NexoBackendApplication.java
│
├── auth/
│   ├── controller/
│   │   └── AuthController.java
│   ├── service/
│   │   ├── AuthService.java
│   │   ├── JwtService.java
│   │   └── VerificationCodeService.java
│   ├── entity/
│   │   └── VerificationCode.java
│   ├── repository/
│   │   └── VerificationCodeRepository.java
│   ├── dto/
│   │   ├── RegisterRequest.java
│   │   ├── LoginRequest.java
│   │   ├── VerifyCodeRequest.java
│   │   └── AuthResponse.java
│   └── config/
│       ├── SecurityConfig.java
│       └── JwtAuthenticationFilter.java
│
├── user/
│   ├── controller/
│   │   ├── UserController.java
```

```
│   │   └── RoleController.java
│   ├── service/
│   │   ├── UserService.java
│   │   └── RoleService.java
│   ├── entity/
│   │   ├── User.java
│   │   └── Role.java
│   ├── repository/
│   │   ├── UserRepository.java
│   │   └── RoleRepository.java
│   └── dto/
│       ├── UserProfileResponse.java
│       ├── UpdateNicknameRequest.java
│       └── AssignRoleRequest.java
│
├── academic/
│   ├── controller/
│   │   └── AcademicController.java
│   ├── service/
│   │   ├── SubjectService.java
│   │   ├── AcademicOfferService.java
│   │   └── RestrictionValidatorService.java
│   ├── entity/
│   │   ├── AcademicOffer.java
│   │   ├── Subject.java
│   │   ├── Group.java
│   │   ├── TimeBlock.java
│   │   ├── StudyPlan.java
│   │   └── Prerequisite.java
│   ├── repository/
│   │   ├── AcademicOfferRepository.java
│   │   ├── SubjectRepository.java
│   │   ├── GroupRepository.java
│   │   ├── TimeBlockRepository.java
│   │   └── StudyPlanRepository.java
│   └── dto/
│       ├── SubjectResponse.java
│       ├── GroupResponse.java
│       └── AcademicSearchRequest.java
│
├── schedule/
│   ├── controller/
│   │   └── ScheduleController.java
│   ├── service/
```

```
│   │   ├── ScheduleService.java
│   │   └── ExportService.java
│   ├── entity/
│   │   ├── Schedule.java
│   │   └── ScheduleBlock.java
│   ├── repository/
│   │   ├── ScheduleRepository.java
│   │   └── ScheduleBlockRepository.java
│   └── dto/
│       ├── ScheduleResponse.java
│       ├── CreateScheduleRequest.java
│       └── AddBlockRequest.java
│
├── campus/
│   ├── controller/
│   │   └── CampusController.java
│   ├── service/
│   │   ├── CampusService.java
│   │   └── TravelTimeService.java
│   ├── entity/
│   │   ├── Campus.java
│   │   ├── Classroom.java
│   │   └── ClassroomPhoto.java
│   ├── repository/
│   │   ├── CampusRepository.java
│   │   └── ClassroomRepository.java
│   └── dto/
│       ├── CampusResponse.java
│       └── TravelTimeResponse.java
│
├── content/
│   ├── controller/
│   │   ├── AnnouncementController.java
│   │   ├── WelfareController.java
│   │   └── CalendarController.java
│   ├── service/
│   │   ├── AnnouncementService.java
│   │   ├── WelfareContentService.java
│   │   └── GeneralCalendarService.java
│   ├── entity/
│   │   ├── Announcement.java
│   │   ├── WelfareContent.java
│   │   └── CalendarEvent.java
│   ├── repository/
```

```
│   │   ├── AnnouncementRepository.java
│   │   ├── WelfareContentRepository.java
│   │   └── CalendarEventRepository.java
│   └── dto/
│       ├── AnnouncementRequest.java
│       ├── AnnouncementResponse.java
│       ├── WelfareContentResponse.java
│       └── CalendarEventResponse.java
│
├── report/
│   ├── controller/
│   │   └── ReportController.java
│   ├── service/
│   │   └── ReportService.java
│   ├── entity/
│   │   └── Report.java
│   ├── repository/
│   │   └── ReportRepository.java
│   └── dto/
│       ├── CreateReportRequest.java
│       └── ReportResponse.java
│
├── admin/
│   ├── controller/
│   │   └── AdminController.java
│   ├── service/
│   │   ├── AcademicLoadService.java
│   │   └── FileValidationService.java
│   └── dto/
│       ├── LoadResultResponse.java
│       └── ValidationErrorResponse.java
│
└── shared/
    ├── exception/
    │   ├── GlobalExceptionHandler.java
    │   ├── ResourceNotFoundException.java
    │   ├── BusinessRuleException.java
    │   └── UnauthorizedException.java
    ├── dto/
    │   ├── ApiResponse.java
    │   └── PageResponse.java
    ├── config/
    │   ├── CorsConfig.java
    │   ├── OpenApiConfig.java
```

```
    │   └── MailConfig.java
    └── util/
        └── DateUtils.java
```

5.3 Directorio resources

```
src/main/resources/
├── application.yml
├── application-dev.yml
├── application-prod.yml
├── db/
│   └── migration/
│       ├── V1__create_users_and_roles.sql
│       ├── V2__create_academic_tables.sql
│       ├── V3__create_schedule_tables.sql
│       ├── V4__create_campus_tables.sql
│       ├── V5__create_content_tables.sql
│       └── V6__create_report_tables.sql
└── templates/
    └── verification-email.html
```

5.4 Directorio de tests

```
src/test/java/com/kumorai/nexo/
├── auth/
│   ├── controller/
│   │   └── AuthControllerTest.java
│   └── service/
│       └── AuthServiceTest.java
├── academic/
│   └── service/
│       └── RestrictionValidatorServiceTest.java
├── schedule/
│   └── service/
│       └── ScheduleServiceTest.java
└── ...  (espejo de la estructura principal)
```

6. Convenciones y Reglas del Proyecto

6.1 Reglas de comunicación entre módulos

Un módulo puede invocar servicios de otro módulo únicamente a través de su interfaz de servicio pública. Nunca se debe inyectar directamente un repositorio de otro módulo. Las relaciones entre entidades de distintos módulos se establecen mediante IDs (claves foráneas a nivel de base de datos), no mediante asociaciones JPA (@ManyToOne, @OneToMany) que crucen fronteras de módulo.

6.2 Convenciones de nombrado

Elemento	Convención	Ejemplo
Paquetes	lowercase, singular	com.kumorai.nexo.auth
Clases	PascalCase	ScheduleService
Métodos	camelCase	findByEmail()
Constantes	UPPER_SNAKE_CASE	MAX_CREDITS
Tablas BD	snake_case, plural	academic_offers
Endpoints	kebab-case, plural	/api/v1/schedules
DTOs	[Nombre]Request / Response	LoginRequest.java

6.3 Estructura de URLs de la API

Todos los endpoints siguen el patrón /api/v1/{modulo}/{recurso}. Los endpoints públicos (accesibles sin autenticación) se agrupan bajo /api/v1/public/. Los endpoints de administrador bajo /api/v1/admin/.

Método	Endpoint	Acceso
POST	/api/v1/auth/register	Público
POST	/api/v1/auth/verify	Público
POST	/api/v1/auth/login	Público
GET	/api/v1/public/academic-offer	Público (planeador sin sesión)
GET	/api/v1/users/me	Estudiante
GET	/api/v1/schedules	Estudiante
POST	/api/v1/schedules	Estudiante
POST	/api/v1/reports	Estudiante
POST	/api/v1/content/announcements	Radicador de Avisos
POST	/api/v1/admin/academic-load	Administrador
PUT	/api/v1/admin/users/{id}/roles	Administrador

7. Perfiles de Configuración

Spring Boot permite separar la configuración por entorno mediante perfiles. El proyecto define tres perfiles:

application.yml: configuración común a todos los entornos (nombres de propiedades, estructura general).

application-dev.yml: base de datos local (localhost:5432/nexo_dev), logs en nivel DEBUG, Swagger habilitado, CORS permisivo para el frontend en localhost:5173.

application-prod.yml: base de datos de producción (variables de entorno), logs en nivel WARN, Swagger deshabilitado, CORS restringido al dominio de producción, HTTPS enforced.

El perfil activo se define mediante la variable de entorno SPRING_PROFILES_ACTIVE o el parámetro --spring.profiles.active=dev al ejecutar el JAR.

8. Mapa de Dependencias entre Módulos

El siguiente mapa describe las dependencias permitidas entre los módulos del sistema. Las flechas indican que un módulo consume servicios del otro:

```
auth ──────▶ user (consultar/crear usuario)
schedule ──▶ academic (validar restricciones, obtener bloques)
schedule ──▶ campus (calcular tiempos de traslado)
schedule ──▶ user (obtener datos del estudiante)
admin ─────▶ academic (cargar oferta académica)
admin ─────▶ user (gestionar roles)
admin ─────▶ content (CRUD de contenido general)
admin ─────▶ campus (gestionar sedes)
content ───▶ user (verificar rol de radicador)
report ────▶ user (asociar reporte al estudiante)
shared ◀── TODOS (componentes transversales)
```

Cualquier dependencia no listada aquí está prohibida. Si surge la necesidad de una nueva dependencia, debe discutirse y documentarse antes de implementarse.

9. Primeros Pasos para el Equipo

Para comenzar a trabajar con el backend del proyecto, el equipo debe seguir esta secuencia:

1. **Instalar los prerequisitos:** JDK 17 (recomendado: Eclipse Temurin o Amazon Corretto), Maven 3.9+, PostgreSQL 16+, y un IDE (IntelliJ IDEA con licencia estudiantil o VS Code con Extension Pack for Java).
2. **Generar el proyecto base** desde Spring Initializr con los parámetros de la sección 4.1, o usar el comando curl de la sección 4.3.
3. **Crear la estructura de paquetes** siguiendo la distribución de la sección 5. Crear los paquetes vacíos con al menos una clase placeholder para que Git los registre.
4. **Configurar la base de datos local:** crear la base de datos nexo_dev en PostgreSQL, configurar las credenciales en application-dev.yml.

5. **Agregar las dependencias manuales** (jjwt, springdoc-openapi, mapstruct) al pom.xml.
6. **Crear el docker-compose.yml** con un servicio de PostgreSQL para que todo el equipo trabaje con la misma versión de base de datos.
7. **Ejecutar ./mvnw spring-boot:run** y verificar que la aplicación inicia correctamente en http://localhost:8080.
8. **Subir al repositorio GitHub** del proyecto con un commit inicial y configurar las ramas según la estrategia del equipo.

Fin del documento